



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería en Software

EXPOSICIÓN SOBRE MODEL-DRIVEN DEVELOPMENT (MDD) CON BOUML

Caso de estudio: Sistema de Gestión Clínica Veterinaria con IA
(SGCV-IA)

Módulo de Gestión de Citas Médicas (Atenciones Clínicas)

Asignatura: Ingeniería de Requisitos (ISR-401)

Docente: Ing. Gleiston Guerrero Ulloa, PhD.

Nivel / Curso: 4to A Software

Período académico: 2026–2027 PPA

INTEGRANTES

Arteaga Danela

Herrera Thais

Marcillo Jeanpool

Robison Jenapierrel

Quevedo – Ecuador

Fecha de entrega: 18 de julio de 2026

Índice

Índice de figuras	4
Índice de cuadros	5
Resumen	6
1 Introducción	7
1.1 Contexto y Motivación	7
1.2 Problemática del Desarrollo Tradicional	7
1.3 Objetivo del Trabajo	7
1.4 Estructura del Documento	7
2 Marco Teórico	9
2.1 Ingeniería Dirigida por Modelos (MDE)	9
2.1.1 Principios Fundamentales de MDE	9
2.2 Model Driven Architecture (MDA)	9
2.2.1 Modelos CIM, PIM y PSM	9
2.3 Model-Driven Development (MDD)	10
2.3.1 Transformaciones Modelo a Modelo (M2M)	10
2.3.2 Transformaciones Modelo a Texto (M2T)	10
2.4 UML (Unified Modeling Language)	10
2.5 Modelado de Requisitos y MDD	10
2.6 Ingeniería Directa, Inversa y Roundtrip	11
2.6.1 Ingeniería Directa (Forward Engineering)	11
2.6.2 Ingeniería Inversa (Reverse Engineering)	11
2.6.3 Roundtrip Engineering	11
2.7 Beneficios y Limitaciones del MDD	11
2.7.1 Beneficios	11
2.7.2 Limitaciones	11
2.8 Estado del Arte	12
3 Justificación de la Elección de BOUML	13
3.1 Descripción de BOUML	13
3.2 Características Principales	13
3.3 Comparación con Otras Herramientas	13
3.4 Por qué BOUML para este Trabajo	13
4 Descripción del PFC y Módulo Seleccionado	15
4.1 Proyecto Fin de Curso: SGCV-IA	15
4.1.1 Objetivo General del PFC	15
4.1.2 Alcance del Sistema	15
4.2 Módulo Seleccionado: Atenciones Clínicas	15
4.2.1 Objetivo del Módulo	15

4.2.2	Alcance del Módulo	16
4.2.3	Actores del Módulo	16
4.2.4	Funcionalidades	16
5	Modelo UML	17
5.1	Diagrama de Clases UML	17
5.2	Clases Principales	18
5.3	Relaciones y Cardinalidades	19
5.4	Diagramas Complementarios	20
5.4.1	Diagrama de Casos de Uso	20
5.4.2	Diagrama de Máquina de Estados	20
6	Configuración del Generador	21
6.1	Configuración de BOUML para C++	21
6.1.1	Lenguajes Soportados por BOUML	21
6.2	Parámetros de Configuración	21
6.3	Configuración de Artefactos	21
7	Generación y Verificación del Código	23
7.1	Proceso de Generación en C++	23
7.1.1	Flujo de Generación	23
7.2	Estructura de Archivos Generados en C++	24
7.3	Verificación del Código Generado	24
7.4	Limitaciones Detectadas	24
8	Roundtrip Engineering	25
8.1	Demostración Práctica en C++	25
8.1.1	Antes del Cambio	25
8.1.2	Pasos del Roundtrip	25
8.1.3	Después del Cambio	25
8.2	Traza del Cambio	25
8.3	Ventajas del Roundtrip con BOUML	26
9	Distribución del Trabajo por Integrante	27
10	Conclusiones	28
10.1	Contribución al PFC	28
10.2	Recomendaciones	29
11	Referencias	30
	Referencias	30
12	Anexos	31
12.1	Anexo A: Código Fuente Generado en C++	31
12.1.1	Archivo de Cabecera (Mascota.h)	31

12.1.2 Archivo de Implementación (Mascota.cpp)	31
12.2 Anexo B: Estructura del Repositorio	33
12.3 Anexo C: Lenguajes Soportados por BOUML	33
12.4 Anexo D: Reunión del trabajo realizado	33

Índice de figuras

1	Diagrama de Clases UML del Módulo de Gestión de Atenciones Clínicas....	17
2	Diagrama de Casos de Uso del Módulo de Atenciones Clínicas	20
3	Diagrama de Máquina de Estados de la entidad AtencionClinica	20
4	Configuración de la Generación de Código en BOUML	22
5	Proceso de Generación de Código en BOUML	23
6	Modelo Inicial de la Clase Mascota	25
7	Código Actualizado de Mascota.cpp	25
8	Captura de GitHub Insights → Contributors	27
9	Estructura del Repositorio en GitHub	33
10	Captura de GitHub Insights → Contributors	34

Índice de cuadros

1	Comparación de herramientas MDD	13
2	Clases del modelo UML	18
3	Relaciones del modelo UML	19
4	Configuración del generador en BOUML para C++	21
5	Correspondencia modelo-código en C++	24
6	Distribución del trabajo por integrante	27
7	Lenguajes soportados por BOUML según captura de pantalla	33

Resumen

El presente trabajo aborda la aplicación de Model-Driven Development (MDD) como enfoque para el desarrollo de software, utilizando la herramienta BOUML para la generación automática de código a partir de modelos UML. El estudio se centra en el módulo de Gestión de Citas Médicas (Atenciones Clínicas) del Sistema de Gestión Clínica Veterinaria con IA (SGCV-IA), correspondiente al Proyecto Fin de Curso (PFC) del equipo. Se presenta el marco teórico que sustenta la Ingeniería Dirigida por Modelos (MDE), la Arquitectura Dirigida por Modelos (MDA), y los conceptos de PIM y PSM. Se justifica la selección de BOUML frente a otras herramientas UML como StarUML, Enterprise Architect y Visual Paradigm, destacando su naturaleza libre, ligereza y capacidades de generación de código. El trabajo incluye un manual detallado del uso de BOUML, el diseño del diagrama de clases del sistema basado en el modelo de datos del SGCV-IA (14 entidades), el proceso de generación de código en C++ (lenguaje soportado por BOUML), y la verificación del código generado. Además, se aborda el concepto de Roundtrip Engineering para mantener la sincronización entre el modelo y el código. El contenido se encuentra disponible en un repositorio público de GitHub, organizado según las buenas prácticas de documentación.

Palabras clave: MDD, BOUML, UML, Generación de Código, Roundtrip Engineering, SGCV-IA, C++.

1 Introducción

1.1 Contexto y Motivación

En el contexto actual de la ingeniería de software, la complejidad de los sistemas y la necesidad de acelerar los ciclos de desarrollo han impulsado la adopción de enfoques que elevan el nivel de abstracción en el proceso de construcción de software. La Ingeniería Dirigida por Modelos (MDE) propone que los modelos, más que el código, sean los artefactos primarios del desarrollo, permitiendo la generación automática de implementaciones a partir de especificaciones formales [1]. Esta filosofía, materializada en la Arquitectura Dirigida por Modelos (MDA) de la Object Management Group (OMG), busca separar la lógica de negocio de los detalles de la plataforma tecnológica, mejorando la portabilidad, la interoperabilidad y la productividad [4].

El Model-Driven Development (MDD) se presenta como una aplicación práctica de estos principios, donde los modelos UML se transforman sistemáticamente en código ejecutable. Este enfoque no solo acelera el desarrollo, sino que también reduce errores y mantiene la trazabilidad entre el diseño y la implementación [2]. En este marco, las herramientas CASE juegan un papel fundamental. BOUML, una herramienta de modelado UML libre y multiplataforma, destaca por su velocidad y capacidad para generar código en múltiples lenguajes, incluyendo C++ [9].

1.2 Problemática del Desarrollo Tradicional

El desarrollo de software tradicional enfrenta diversos desafíos que el MDD busca resolver:

- **Brecha semántica:** Dificultad para mantener la coherencia entre los requisitos, el diseño y la implementación, lo que genera inconsistencias y errores difíciles de rastrear.
- **Documentación obsoleta:** Los documentos de diseño tienden a desactualizarse rápidamente, perdiendo su valor como guía para el desarrollo y mantenimiento.
- **Productividad limitada:** El código repetitivo (getters, setters, constructores, etc.) consume tiempo valioso que podría dedicarse a la lógica de negocio.
- **Errores de implementación:** La traducción manual del diseño al código introduce errores humanos, especialmente en sistemas complejos.

1.3 Objetivo del Trabajo

El presente trabajo se enmarca en la asignatura de Ingeniería de Requisitos y tiene como objetivo demostrar la aplicación práctica de MDD utilizando BOUML, tomando como caso de estudio el módulo de Gestión de Citas Médicas del Sistema de Gestión Clínica Veterinaria con IA (SGCV-IA), que constituye el Proyecto Fin de Curso (PFC) del equipo.

1.4 Estructura del Documento

Este documento se organiza en las siguientes secciones: Marco Teórico, Justificación de la Herramienta, Descripción del PFC, Modelo UML, Configuración del Generador,

Proceso de Generación, Verificación del Código, Roundtrip Engineering, Distribución del Trabajo, Conclusiones y Referencias.

2 Marco Teórico

2.1 Ingeniería Dirigida por Modelos (MDE)

La Ingeniería Dirigida por Modelos (Model-Driven Engineering - MDE) es un paradigma de desarrollo de software que utiliza modelos como los artefactos principales en todas las fases del ciclo de vida del software [1].

2.1.1 Principios Fundamentales de MDE

1. **Modelos como artefactos de primera clase:** Los modelos no son solo documentación, sino que son la fuente autoritativa del sistema. Son elementos ejecutables o transformables que guían todo el proceso de desarrollo.
2. **Transformaciones automatizadas:** La generación de código y otros artefactos se realiza mediante transformaciones automáticas, reduciendo la intervención manual y los errores asociados.
3. **Abstracción:** Los modelos permiten trabajar con conceptos del dominio sin preocuparse por detalles de implementación, facilitando la comunicación entre stakeholders.

El objetivo central de la MDE es elevar el nivel de abstracción en el desarrollo, permitiendo a los ingenieros trabajar con conceptos del dominio del problema en lugar de lidiar con los detalles de implementación de una plataforma específica. Este enfoque promete mejorar la productividad y la calidad del software [3].

2.2 Model Driven Architecture (MDA)

La Arquitectura Dirigida por Modelos (Model Driven Architecture - MDA) es una iniciativa de la OMG lanzada en 2001 para estandarizar y guiar la aplicación de la MDE [4]. Sus pilares fundamentales son:

- **Meta-Object Facility (MOF):** Define la infraestructura para crear metamodelos, proporcionando un estándar para la definición de lenguajes de modelado.
- **Unified Modeling Language (UML):** Lenguaje de modelado visual estándar para la especificación, visualización, construcción y documentación de sistemas de software.
- **XML Metadata Interchange (XMI):** Estándar para el intercambio de modelos entre diferentes herramientas, facilitando la interoperabilidad.

2.2.1 Modelos CIM, PIM y PSM

La arquitectura MDA se estructura en torno a tres niveles de abstracción [4]:

1. **Computation Independent Model (CIM):** Describe los requisitos del sistema y la lógica de negocio sin considerar su implementación tecnológica. Se enfoca en el "qué" debe hacer el sistema y es comprensible para los expertos del dominio.

2. **Platform Independent Model (PIM):** Es un modelo del sistema que especifica su funcionalidad y estructura, pero sin detalles de la plataforma tecnológica. Representa el cómo.^a nivel de diseño y es independiente de la implementación.
3. **Platform Specific Model (PSM):** Es un modelo del sistema adaptado a una plataforma tecnológica concreta. Añade detalles de implementación específicos como el framework de persistencia, el middleware o el lenguaje de programación.

2.3 Model-Driven Development (MDD)

El Model-Driven Development (MDD) es la aplicación práctica de los principios de MDE/MDA en el proceso de desarrollo de software [2]. El proceso de MDD se apoya en dos tipos fundamentales de transformaciones [7]:

2.3.1 Transformaciones Modelo a Modelo (M2M)

Las transformaciones M2M producen un modelo destino a partir de uno de origen, ambos conformes a metamodelos. El estándar OMG de referencia es QVT (Query/View/-Transformation) [6]. Un ejemplo típico es la transformación de un PIM a un PSM, donde se añaden detalles de implementación específicos de la plataforma.

2.3.2 Transformaciones Modelo a Texto (M2T)

Las transformaciones M2T producen artefactos textuales (código fuente, configuraciones, documentación) a partir de un modelo. El estándar OMG de referencia es MOFM2T (MOF Model To Text) [7]. Estas transformaciones son el paso final que genera código ejecutable a partir del modelo.

2.4 UML (Unified Modeling Language)

UML es un lenguaje de modelado visual de propósito general, estandarizado por la OMG [5]. Proporciona una notación gráfica para representar diferentes aspectos de un sistema a través de diversos tipos de diagramas:

- **Diagramas Estructurales:** Clases, Componentes, Despliegue, Objetos, Paquetes, Estructura Compuesta, Perfiles.
- **Diagramas de Comportamiento:** Casos de Uso, Actividades, Máquinas de Estados, Secuencia, Comunicación, Tiempo, Interacción General.

En el contexto de MDD, UML es el lenguaje principal para construir los modelos PIM y PSM [3].

2.5 Modelado de Requisitos y MDD

La especificación de requisitos se rige por la norma IEEE/ISO/IEEE 29148:2018 [8]. Esta norma establece que la calidad de un SRS se evalúa según cuatro atributos: completitud, consistencia, verificabilidad y trazabilidad.

Las tres perspectivas de modelado son complementarias y ninguna es suficiente por sí sola [3]:

- **Modelo de Datos:** Responde a "¿qué existe en el sistema?". Representa las entidades y sus relaciones a través de diagramas ER y de clases.

- **Modelo Funcional:** Responde a "¿cómo se transforman esos datos?". Representa los procesos y flujos de datos mediante DFD y diagramas de actividades.
- **Modelo de Comportamiento:** Responde a "¿cuándo y en qué orden ocurren esas transformaciones?". Representa los estados y transiciones mediante máquinas de estados y diagramas de secuencia.

2.6 Ingeniería Directa, Inversa y Roundtrip

La gestión de la relación entre el modelo y el código se aborda mediante tres conceptos fundamentales [3]:

2.6.1 Ingeniería Directa (Forward Engineering)

Es el proceso tradicional de generar código a partir de un modelo. Se parte del diseño UML y se obtiene el código fuente en el lenguaje de programación deseado. Es el flujo principal en MDD y permite mantener la coherencia entre el diseño y la implementación.

2.6.2 Ingeniería Inversa (Reverse Engineering)

Es el proceso inverso. Se parte del código fuente existente y se genera o actualiza un modelo UML que lo represente. Es útil para documentar sistemas legados o para incorporar código escrito manualmente al proceso de modelado.

2.6.3 Roundtrip Engineering

El Roundtrip Engineering es la capacidad de una herramienta de soportar tanto la ingeniería directa como la inversa de manera iterativa, manteniendo la consistencia entre el modelo y el código [3]. Su característica clave es la sincronización incremental: no se trata simplemente de regenerar todo, sino de fusionar los cambios realizados en el modelo o en el código sin perder el trabajo realizado en el otro artefacto.

2.7 Beneficios y Limitaciones del MDD

2.7.1 Beneficios

- **Aumento de la productividad:** Automatización de tareas repetitivas, reduciendo el tiempo de desarrollo.
- **Mejora de la calidad:** El código generado es consistente con el modelo, reduciendo errores.
- **Portabilidad:** El mismo PIM puede generar múltiples PSMs para diferentes plataformas.
- **Documentación viva:** El modelo se mantiene sincronizado con el código, evitando la obsolescencia.
- **Trazabilidad:** Facilita el rastreo entre requisitos, diseño y código.

2.7.2 Limitaciones

- **Curva de aprendizaje:** Requiere formación en herramientas y conceptos de modelado.
- **Madurez de herramientas:** No todas las herramientas soportan todas las funcionalidades necesarias.

- **Personalización:** El código generado puede ser rígido y difícil de modificar.
- **Sobrecarga inicial:** Inversión de tiempo en modelos formales que puede no ser rentable en proyectos pequeños.

2.8 Estado del Arte

El campo de la transformación de modelos cuenta con más de 60 herramientas catalogadas [12]. Los principales desafíos vigentes incluyen:

- **Escalabilidad:** Manejo de modelos de gran tamaño con miles de elementos.
- **Interoperabilidad:** Intercambio de modelos entre diferentes herramientas mediante XMI.
- **Adopción industrial:** Integración con flujos de trabajo y herramientas existentes.
- **Mantenibilidad:** Evolución de modelos y transformaciones a lo largo del tiempo [10].

3 Justificación de la Elección de BOUML

3.1 Descripción de BOUML

BOUML es una herramienta de modelado UML 2.0 de código abierto, desarrollada por Bruno Pagès y publicada bajo la Licencia Pública General de GNU (GPLv2) [9]. Fue diseñada con un enfoque en el rendimiento y la eficiencia, compitiendo favorablemente con herramientas comerciales más pesadas.

3.2 Características Principales

- **Código Abierto y Gratuito:** Accesible para entornos académicos y profesionales sin coste de licencia.
- **Multiplataforma:** Funciona en Windows, Linux, Solaris y Mac OS X.
- **Ultra-ligero:** Bajo consumo de memoria y alta velocidad, permitiendo trabajar con modelos de gran tamaño.
- **Soporte UML 2.0:** Cubre una amplia gama de diagramas UML.
- **Generación de Código:** Soporta C++, Java, Python, PHP, IDL y SQL.
- **Ingeniería Inversa:** Capacidad de recuperar modelos desde código existente.
- **Extensible:** Permite el desarrollo de plugins mediante API en C++ o Java.
- **Multi-usuario:** Soporte para trabajo colaborativo con control de versiones.

3.3 Comparación con Otras Herramientas

Cuadro 1: Comparación de herramientas MDD

Característica	BOUML	StarUML	Visual Paradigm
Licencia	Gratuito (GPL)	Freemium	De pago
Rendimiento	Muy Alto	Bajo	Medio
Lenguajes Generados	C++, Java, Python, PHP, SQL	Java, C#, Python	Java, C#, PHP, Python
Colaboración	No	No	Sí
Extensibilidad	Sí	Sí	Sí
Simulación	No	No	Sí
Validación	No	No	Sí (OCL)

3.4 Por qué BOUML para este Trabajo

La selección de BOUML se justifica por las siguientes razones:

1. **Accesibilidad:** Coste cero para todos los integrantes del equipo, sin barreras económicas.

2. **Enfoque en MDD:** Flujo de trabajo claro que separa PIM de PSM, alineado con los conceptos teóricos.
3. **Velocidad:** Permite iteraciones rápidas y regeneraciones ágiles, crucial para la demostración.
4. **Soporte para C++:** Lenguaje nativo soportado por BOUML, alineado con los objetivos del proyecto.
5. **Facilidad de demostración:** Interfaz clara y directa para la exposición en clase.

4 Descripción del PFC y Módulo Seleccionado

4.1 Proyecto Fin de Curso: SGCV-IA

El Proyecto Fin de Curso consiste en el desarrollo de un **Sistema de Gestión Clínica Veterinaria con Inteligencia Artificial (SGCV-IA)**. El sistema está diseñado para optimizar y digitalizar los procesos administrativos y clínicos de una clínica veterinaria, integrando capacidades de inteligencia artificial para el apoyo diagnóstico.

4.1.1 Objetivo General del PFC

Desarrollar un sistema de software robusto, seguro y escalable que permita la gestión centralizada de pacientes (mascotas), clientes (dueños), historiales clínicos, citas, inventario y personal veterinario, facilitando la toma de decisiones y mejorando la experiencia de los usuarios.

4.1.2 Alcance del Sistema

El sistema abarcará múltiples módulos interconectados:

- **Módulo de Gestión de Usuarios y Accesos (RBAC):** Administración de usuarios con control de acceso basado en roles, diferenciando entre Administradores, Veterinarios y Personal Administrativo.
- **Módulo de Gestión de Clientes y Mascotas:** Registro, actualización y consulta de información de dueños y sus mascotas.
- **Módulo de Gestión de Citas y Atenciones Clínicas:** Programación, modificación y cancelación de citas, así como el registro detallado de atenciones.
- **Módulo de Inteligencia Artificial:** Generación de sugerencias diagnósticas asistidas por IA, con validación obligatoria por parte del veterinario.
- **Módulo de Gestión de Inventario:** Control de stock de productos, alertas de vencimiento y umbrales mínimos.
- **Módulo de Facturación y Reportes:** Procesamiento de cobros, generación de facturas y reportes administrativos.
- **Módulo de Notificaciones:** Envío de recordatorios, alertas y comunicaciones a los dueños de mascotas.

4.2 Módulo Seleccionado: Atenciones Clínicas

Para la presente exposición, se ha seleccionado el **Módulo de Gestión de Citas Médicas (Atenciones Clínicas)**. Este módulo es crítico en cualquier sistema de salud veterinaria, ya que gestiona la interacción directa entre el paciente, el cliente y el servicio médico.

4.2.1 Objetivo del Módulo

Gestionar de manera eficiente y sin conflictos la agenda de los veterinarios, permitiendo a los clientes solicitar, reprogramar y cancelar citas para sus mascotas, y a los administradores controlar la disponibilidad horaria.

4.2.2 Alcance del Módulo

El módulo se encargará de:

- Agendar nuevas atenciones clínicas verificando la disponibilidad del veterinario y el horario.
- Modificar los detalles de citas existentes (fecha, hora, motivo).
- Cancelar citas y liberar espacios en la agenda.
- Registrar atenciones completas con diagnóstico, tratamiento y medicamentos prescritos.
- Consultar historial de citas por mascota o cliente.
- Integrar diagnóstico asistido por IA durante la atención.
- Gestionar los estados de la atención a lo largo de su ciclo de vida.

4.2.3 Actores del Módulo

- **Veterinario:** Consulta agenda, registra atenciones, valida sugerencias de IA.
- **Administrador/Secretario:** Gestiona citas, agendas, clientes y mascotas.
- **Dueño de Mascota (Cliente):** Solicita y consulta citas.
- **Sistema (Módulo IA):** Genera sugerencias diagnósticas y notificaciones.

4.2.4 Funcionalidades

1. Registrar Atención Clínica
2. Consultar Agenda
3. Modificar Cita
4. Cancelar Cita
5. Ver Historial de Citas
6. Integración con IA
7. Gestión de Estados
8. Reporte de Atenciones

5 Modelo UML

5.1 Diagrama de Clases UML

El diagrama de clases contiene más de 6 clases del dominio del PFC, con atributos tipados, operaciones con firma completa, visibilidad y cardinalidades correctas.

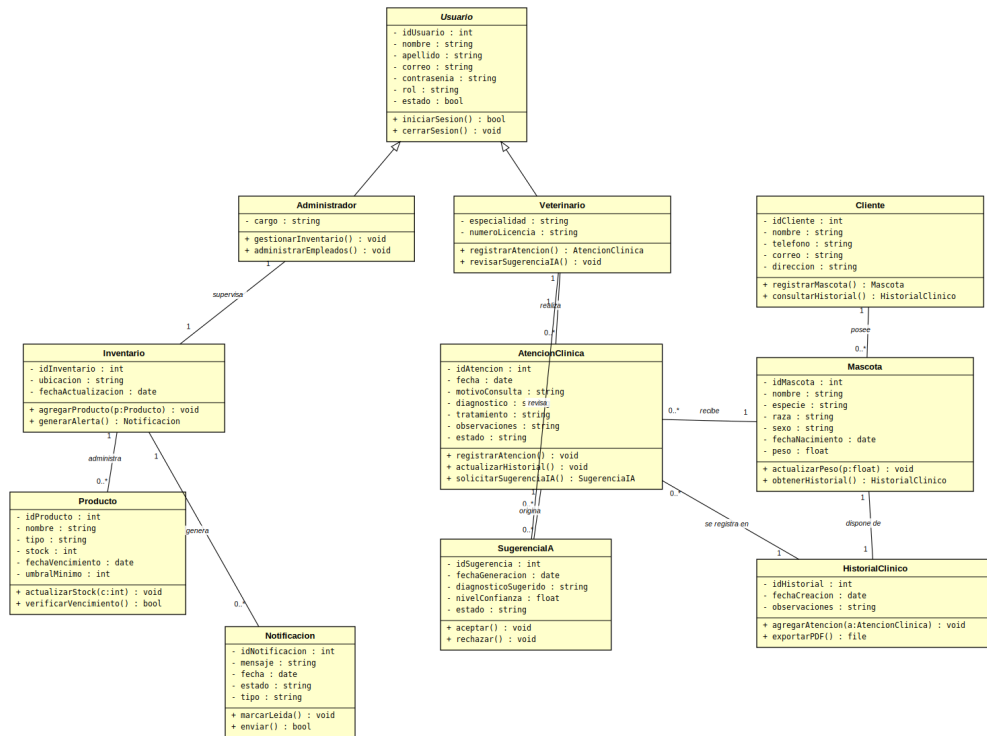


Figura 1: Diagrama de Clases UML del Módulo de Gestión de Atenciones Clínicas

5.2 Clases Principales

Cuadro 2: Clases del modelo UML

Clase	Descripción
Usuario	Clase base abstracta para Administrador y Veterinario . Contiene atributos comunes como idUsuario, nombre, apellido, correo, contrasena, rol y estado.
Administrador	Personal con permisos de administración. Hereda de Usuario y añade el atributo cargo. Métodos: gestionarUsuarios() y generarReporte().
Veterinario	Profesional que atiende a las mascotas. Hereda de Usuario y añade especialidad y numeroLicencia. Métodos: consultarAgenda(), registrarAtencion() y validarSugerenciaIA().
Cliente	Dueño de la mascota. Atributos: idCliente, nombre, telefono, correo, direccion. Métodos: solicitarCita() y consultarHistorialMascotas().
Mascota	Paciente que recibe atención médica. Atributos: idMascota, nombre, especie, raza, sexo, fechaNacimiento, peso. Métodos: actualizarPeso() y consultarHistorialClinico().
AtencionClinica	Evento de consulta médica (clase central). Atributos: idAtencion, fecha, motivoConsulta, diagnostico, tratamiento, observaciones, estado. Métodos: iniciarAtencion(), solicitarDiagnosticoIA(), finalizarAtencion(), cancelarAtencion(), actualizarHistorial().
HistorialClinico	Registro médico completo de la mascota. Atributos: idHistorial, fechaCreacion, observaciones. Método: agregarRegistro().
SugerenciaIA	Sugerencia diagnóstica generada por IA. Atributos: idSugerencia, fechaGeneracion, diagnosticoSugerido, nivelConfianza, estado. Métodos: aceptar(), rechazar(), modificar().
Producto	Medicamentos e insumos. Atributos: idProducto, nombre, tipo, stock, fechaVencimiento, umbralMinimo. Métodos: descontarStock() y verificarVencimiento().
Inventario	Gestiona el inventario de productos. Atributos: idInventario, ubicacion, fechaActualizacion. Método: actualizarStock().
Notificacion	Notificaciones y alertas del sistema. Atributos: idNotificacion, mensaje, fecha, estado, tipo. Método: marcarComoVista().

5.3 Relaciones y Cardinalidades

Cuadro 3: Relaciones del modelo UML

Relación	Tipo	Cardinalidad
Administrador → Usuario	Herencia	-
Veterinario → Usuario	Herencia	-
Cliente Mascota	Asociación	1 1..*
Mascota AtencionClinica	Asociación	1 0..*
Veterinario AtencionClinica	Asociación	1 0..*
Mascota → HistorialClinico	Composición	1 → 1
Inventario → Producto	Agregación	1 → 0..*
Administrador Inventario	Asociación	1 1
AtencionClinica SugerenciaIA	Asociación	1 0..1
Veterinario SugerenciaIA	Asociación	1 0..*
Cliente HistorialClinico	Asociación	1 0..*
Inventario Notificacion	Asociación	1 0..*

5.4 Diagramas Complementarios

5.4.1 Diagrama de Casos de Uso

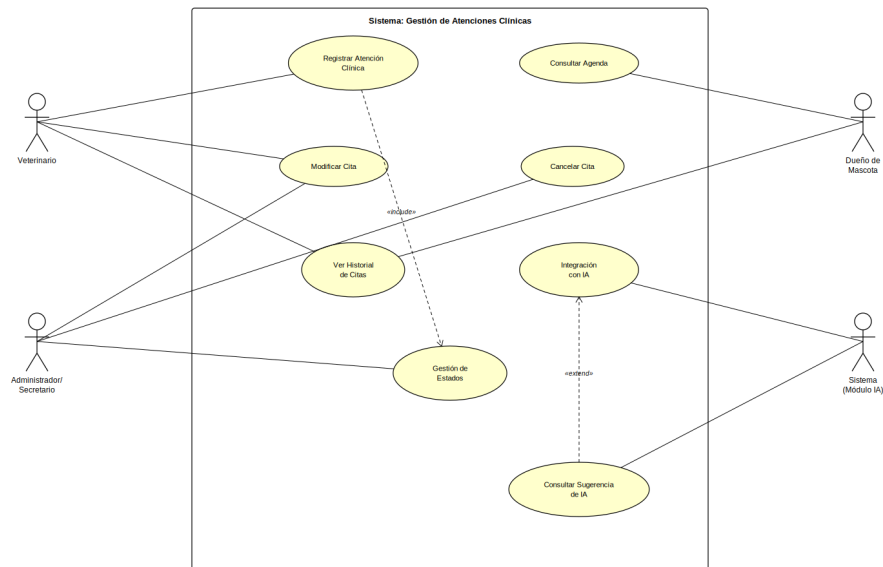


Figura 2: Diagrama de Casos de Uso del Módulo de Atenciones Clínicas

5.4.2 Diagrama de Máquina de Estados

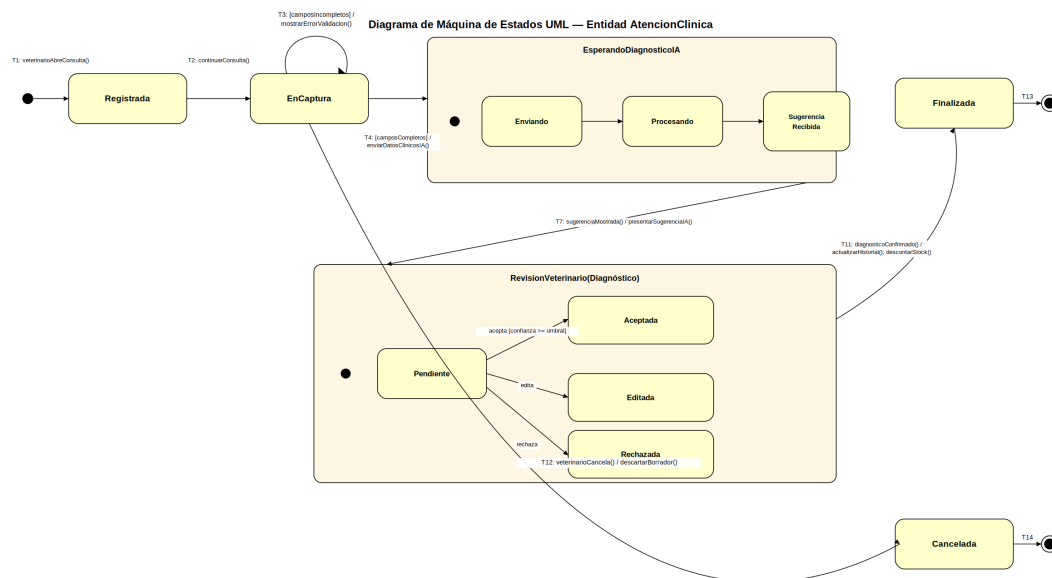


Figura 3: Diagrama de Máquina de Estados de la entidad AtencionClinica

6 Configuración del Generador

6.1 Configuración de BOUML para C++

La generación de código en BOUML se configura a través del menú **Project >Edit Generation Settings**. En la ventana de configuración, se debe seleccionar el lenguaje C++ en la pestaña correspondiente.

6.1.1 Lenguajes Soportados por BOUML

Según la captura de pantalla de la herramienta, BOUML soporta los siguientes lenguajes:

- C++ - Gestión de declaraciones/definiciones por defecto
- Java - Gestión y definición por defecto
- PHP - Gestión y definición por defecto
- Python - Gestión y definición por defecto
- IDL - Gestión de declaraciones por defecto
- MySQL - Gestión y definición por defecto

Nota: C# no está disponible en BOUML, por lo que se ha seleccionado C++ como lenguaje objetivo.

6.2 Parámetros de Configuración

Cuadro 4: Configuración del generador en BOUML para C++

Parámetro	Configuración
Default target language	C++
Default source directory	./CodigoGenerado
Verbose code generation	Activado
Preserve operations's body	Activado
Add operation profile on body edition	Activado
Política de regeneración	Actualización incremental

6.3 Configuración de Artefactos

Se creó una Deployment View llamada **DeploySGCVIA** y se asoció a la Class View **VistaSGCVIA**. Cada clase tiene un artefacto asociado que genera un archivo **.h** (cabecera) y **.cpp** (implementación). Este paso es fundamental para que BOUML sepa qué código debe generar y dónde ubicarlo.

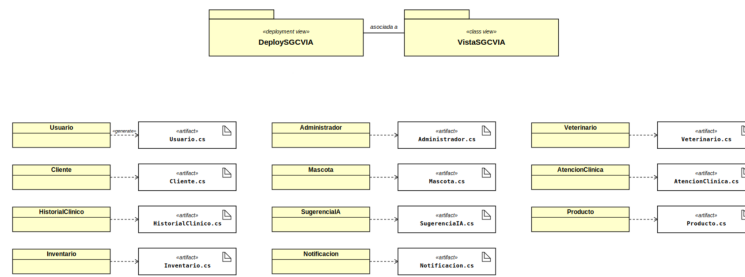


Figura 4: Configuración de la Generación de Código en BOUML

7 Generación y Verificación del Código

7.1 Proceso de Generación en C++

La generación se ejecuta desde la vista de despliegue seleccionando **Generate >C++**. BOUML procesa el modelo y genera los archivos **.h** (declaraciones) y **.cpp** (definiciones) correspondientes.

7.1.1 Flujo de Generación

1. **Análisis del modelo:** BOUML recorre todas las clases, atributos, métodos y relaciones definidas en el diagrama de clases.
2. **Aplicación de configuración:** Se aplican las reglas de generación configuradas en **„Edit Generation Settings”**.
3. **Generación de archivos:** Se escriben los archivos **.h** y **.cpp** en el directorio de salida.
4. **Validación:** Se verifica que el código generado sea sintácticamente correcto.

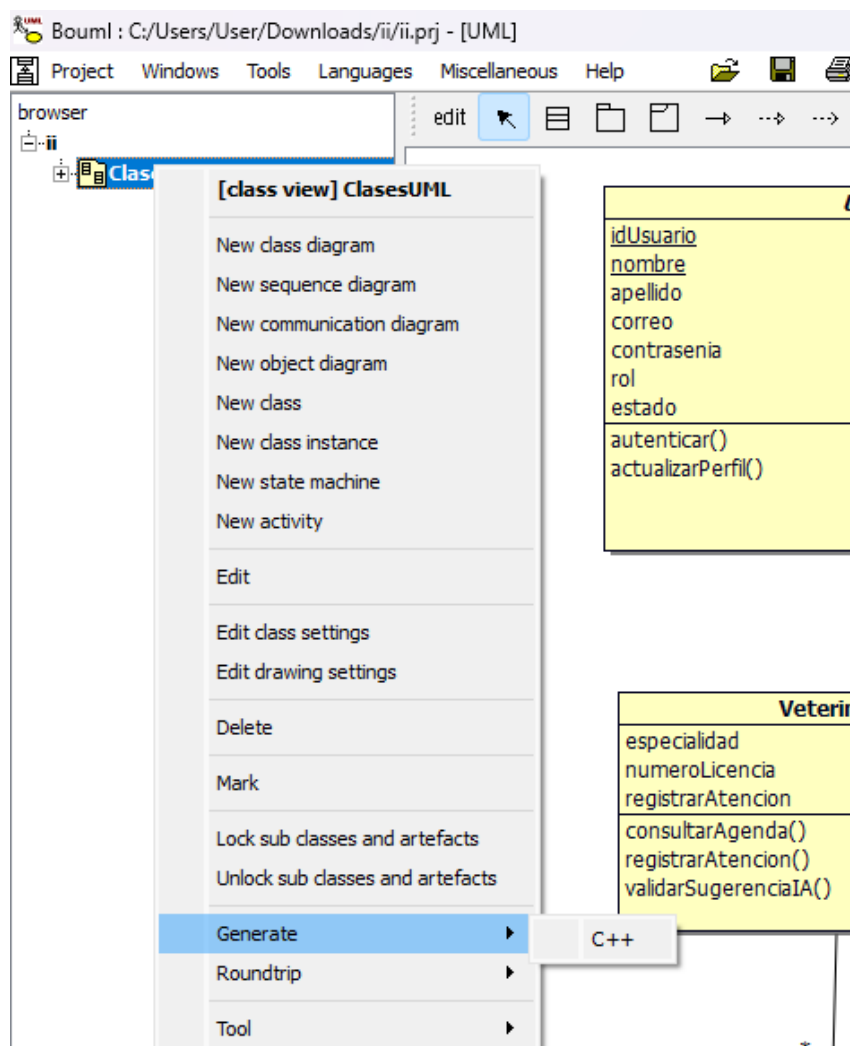


Figura 5: Proceso de Generación de Código en BOUML

7.2 Estructura de Archivos Generados en C++

Para cada clase, BOUML genera dos archivos:

- **Archivo .h (Header):** Contiene la declaración de la clase, atributos, métodos y relaciones.
- **Archivo .cpp (Implementation):** Contiene la implementación de los métodos (con cuerpos vacíos por defecto).

7.3 Verificación del Código Generado

El código generado fue compilado en un entorno C++ (Visual Studio, GCC o similar) sin errores. La siguiente tabla muestra la correspondencia entre los elementos del modelo y los artefactos generados:

Cuadro 5: Correspondencia modelo-código en C++

Clase UML	Archivo Generado	Contenido
Usuario	Usuario.h / .cpp	Clase abstracta con atributos y métodos
Administrador	Administrador.h / .cpp	Hereda de Usuario, atributo cargo
Veterinario	Veterinario.h / .cpp	Hereda de Usuario, atributos especialidad, numeroLicencia
Cliente	Cliente.h / .cpp	Atributos nombre, telefono, correo, direccion
Mascota	Mascota.h / .cpp	Atributos nombre, especie, raza, peso
AtencionClinica	AtencionClinica.h / .cpp	Atributos fecha, motivoConsulta, diagnostico, estado
HistorialClinico	HistorialClinico.h / .cpp	Atributos fechaCreacion, observaciones
SugerenciaIA	SugerenciaIA.h / .cpp	Atributos diagnosticoSugerido, nivelConfianza, estado

7.4 Limitaciones Detectadas

- Los cuerpos de los métodos se generan vacíos (con comentario para implementar)
- No se genera automáticamente la lógica de negocio
- Se requiere implementar manualmente la lógica de validación y persistencia
- La gestión de colecciones (List, Vector) debe ser configurada manualmente en las plantillas de C++
- Las relaciones de composición y agregación no se traducen automáticamente a punteros inteligentes

8 Roundtrip Engineering

8.1 Demostración Práctica en C++

Se ejecutó un cambio significativo en el modelo: la adición del atributo **peso** a la clase Mascota.

8.1.1 Antes del Cambio

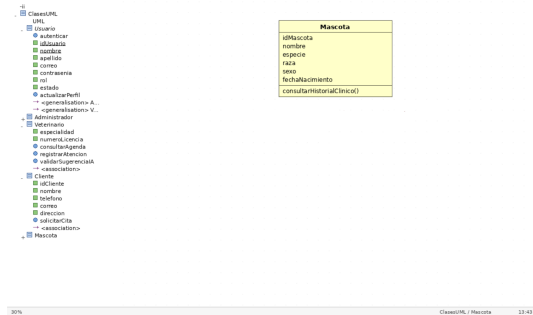


Figura 6: Modelo Inicial de la Clase Mascota

8.1.2 Pasos del Roundtrip

1. **Modificación del modelo:** Agregar atributo `peso: float` a la clase `Mascota` en BOUML.
2. **Regeneración del código:** Ejecutar `Generate >C++` en la Deployment View.
3. **Verificación del cambio:** Los archivos `Mascota.h` y `Mascota.cpp` se actualizan automáticamente.

8.1.3 Después del Cambio

Se agregó el atributo **peso** al modelo y se regeneró el código. Los archivos **Mascota.h** y **Mascota.cpp** se actualizaron automáticamente con la nueva declaración e implementación.

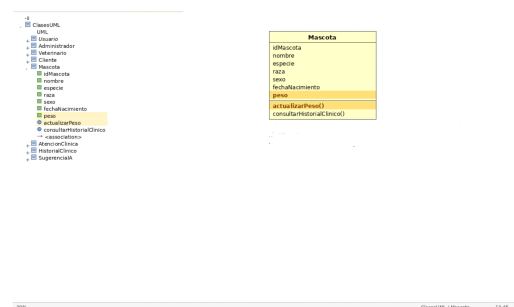


Figura 7: Código Actualizado de Mascota.cpp

8.2 Traza del Cambio

El cambio en el modelo se reflejó en el código sin perder el trabajo manual implementado en los métodos. BOUML actualizó los archivos añadiendo:

1. El nuevo atributo en la sección privada de la clase.
2. Los métodos getter y setter correspondientes.
3. Las declaraciones necesarias en el archivo de cabecera.

8.3 Ventajas del Roundtrip con BOUML

- **Preservación del código manual:** BOUML respeta el código escrito manualmente en los métodos.
- **Actualización incremental:** Solo se modifican los elementos cambiados, no todo el archivo.
- **Documentación viva:** El modelo y el código siempre están sincronizados.

9 Distribución del Trabajo por Integrante

Cuadro 6: Distribución del trabajo por integrante

Integrante	Tareas
Robison Jena-pierrel	Marco teórico, referencias académicas, repositorio, guion de exposición (parte 1)
Arteaga Dane-la	Introducción, descripción del PFC, conclusiones, anexos, referencias IEEE, diapositivas, guion (parte 4)
Herrera Thais	Instalación BOUML, diagrama de clases UML, manual de BOUML, capturas, justificación de herramienta, guion (parte 2)
Marcillo Jean-pool	Generación de código en C++, verificación en Visual Studio, Roundtrip Engineering, tabla de verificación, guion (parte 3)

Commits en GitHub: Cada integrante realizó commits diferenciados en el repositorio, con identidad Git verificable.



Figura 8: Captura de GitHub Insights → Contributors

10 Conclusiones

La realización de este trabajo de exposición sobre Model-Driven Development (MDD) con BOUML ha permitido consolidar los conceptos teóricos de la asignatura de Ingeniería de Requisitos a través de una aplicación práctica.

Principales Aprendizajes:

1. **Valor del MDD:** La separación de la lógica de negocio (PIM) de los detalles de implementación (PSM) a través de modelos UML facilita la portabilidad y el mantenimiento del software [1,2]. La generación automática de código reduce errores y acelera el desarrollo, permitiendo a los desarrolladores centrarse en la lógica de negocio en lugar de en código repetitivo.
2. **BOUML como Herramienta Efectiva:** A pesar de su interfaz austera, BOUML se ha mostrado como una herramienta poderosa, eficiente y ágil para el modelado UML y la generación de código en C++. Su velocidad y bajo consumo de recursos permiten iterar rápidamente, lo que es ideal para un entorno académico y de demostración.
3. **Importancia del Roundtrip:** La capacidad de mantener sincronizados el modelo y el código es fundamental para que el modelo no se convierta en una pieza de documentación obsoleta [3]. El Roundtrip Engineering convierte al modelo en un "contrato vivo" que guía la evolución del sistema, evitando la "deriva modelo-código" manteniendo la trazabilidad.
4. **Aplicación en un Contexto Real:** El uso del módulo de Gestión de Atenciones Clínicas del SGCV-IA ha demostrado la aplicabilidad de MDD a un dominio con una lógica de negocio bien definida. La integración de conceptos como el modelo de datos (14 entidades), el diagrama de casos de uso y la máquina de estados ha evidenciado que los conceptos aprendidos en el aula trascienden los ejemplos académicos.
5. **Desafíos del MDD:** La curva de aprendizaje inicial y la necesidad de invertir tiempo en modelos formales son barreras que deben considerarse. Sin embargo, los beneficios en productividad, calidad y mantenibilidad a largo plazo justifican su adopción, especialmente en sistemas complejos.
6. **Limitación de Lenguajes:** BOUML no soporta C#, por lo que se utilizó C++ como lenguaje objetivo, demostrando la flexibilidad de la herramienta para trabajar con múltiples lenguajes, pero también evidenciando la necesidad de verificar la compatibilidad antes de seleccionar la herramienta.

10.1 Contribución al PFC

Este trabajo sienta las bases para la integración del enfoque MDD en el desarrollo del SGCV-IA. La generación automática de código a partir del modelo UML permitirá mantener la consistencia entre el diseño y la implementación, acelerar el desarrollo de nuevos módulos, facilitar el mantenimiento y la evolución del sistema, y reducir la introducción de errores humanos.

10.2 Recomendaciones

1. Evaluar herramientas alternativas que soporten C# para futuros proyectos.
2. Profundizar en transformaciones M2M utilizando QVT para transformaciones más complejas.
3. Implementar pruebas automatizadas para validar el código generado.
4. Personalizar las plantillas de generación para adaptarlas a las necesidades del proyecto.

11 Referencias

Referencias

- [1] D. C. Schmidt, “Guest editor’s introduction: Model-driven engineering,” *Computer*, vol. 39, no. 2, pp. 25–31, Feb. 2006, doi: 10.1109/MC.2006.58.
- [2] B. Selic, “The pragmatics of model-driven development,” *IEEE Software*, vol. 20, no. 5, pp. 19–25, Sep. 2003, doi: 10.1109/MS.2003.1231146.
- [3] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2nd ed. Berlin: Springer, 2025.
- [4] Object Management Group, “Model Driven Architecture (MDA) Guide,” OMG, Tech. Rep. ormsc/14-06-01, Rev. 2.0, 2014.
- [5] Object Management Group, “OMG Unified Modeling Language (OMG UML),” OMG, Tech. Rep. formal/17-12-05, Version 2.5.1, 2017.
- [6] Object Management Group, “Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification,” OMG, Tech. Rep. formal/16-06-03, Version 1.3, 2016.
- [7] Object Management Group, “MOF Model To Text Transformation Language (MOFM2T),” OMG, Tech. Rep. formal/2008-01-16, Version 1.0, 2008.
- [8] ISO/IEC/IEEE, “Systems and software engineering — Life cycle processes — Requirements engineering,” ISO/IEC/IEEE 29148:2018, 2018.
- [9] B. Pagès, *BOUML: A free UML 2 tool box*, 2024. [Online]. Available: <http://bouml.free.fr/>. Accessed: Jun. 2024.
- [10] A. Bucchiarone, J. Cabot, R. F. Paige, and A. Pierantonio, “Grand challenges in model-driven engineering: an analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 5–13, 2020, doi: 10.1007/s10270-019-00773-6.
- [11] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*, 2nd ed., Synthesis Lectures on Software Engineering. Morgan & Claypool Publishers, 2017, doi: 10.2200/S00751ED2V01Y201701SWE001.
- [12] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varro, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, no. 4, pp. 2361–2397, 2019, doi: 10.1007/s10270-018-0665-6.
- [13] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, v4.0, 2024.

12 Anexos

12.1 Anexo A: Código Fuente Generado en C++

12.1.1 Archivo de Cabecera (Mascota.h)

```

1 // Generated by BOUML, version: 6.7.9
2 // Generated from "ModeloAtenciones", on: 2024-07-18
3
4 #ifndef MASCOTA_H
5 #define MASCOTA_H
6
7 #include <string>
8 #include "HistorialClinico.h"
9
10 class Mascota
11 {
12 private:
13     std::string especie;
14     std::string nombre;
15     float peso;
16     std::string raza;
17     std::string sexo;
18     std::string fechaNacimiento;
19
20 public:
21     Mascota();
22     ~Mascota();
23
24     // Getters
25     std::string getEspecie() const;
26     std::string getNombre() const;
27     float getPeso() const;
28     std::string getRaza() const;
29     std::string getSexo() const;
30     std::string getFechaNacimiento() const;
31
32     // Setters
33     void setEspecie(const std::string& value);
34     void setNombre(const std::string& value);
35     void setPeso(float value);
36     void setRaza(const std::string& value);
37     void setSexo(const std::string& value);
38     void setFechaNacimiento(const std::string& value);
39
40     // M todos
41     void actualizarPeso(float nuevoPeso);
42     HistorialClinico* consultarHistorialClinico();
43 };
44
45 #endif // MASCOTA_H

```

Listing 1: Código generado para la clase Mascota - Mascota.h

12.1.2 Archivo de Implementación (Mascota.cpp)

```

1 // Generated by BOUML, version: 6.7.9
2 // Generated from "ModeloAtenciones", on: 2024-07-18
3
4 #include "Mascota.h"
5 #include "HistorialClinico.h"
6
7 Mascota::Mascota()
8 {
9     // Constructor vacio
10 }

```



```
11
12 Mascota::~Mascota()
13 {
14     // Destructor
15 }
16
17 // Getters
18 std::string Mascota::getEspecie() const
19 {
20     return this->especie;
21 }
22
23 std::string Mascota::getNombre() const
24 {
25     return this->nombre;
26 }
27
28 float Mascota::getPeso() const
29 {
30     return this->peso;
31 }
32
33 // Setters
34 void Mascota::setEspecie(const std::string& value)
35 {
36     this->especie = value;
37 }
38
39 void Mascota::setNombre(const std::string& value)
40 {
41     this->nombre = value;
42 }
43
44 void Mascota::setPeso(float value)
45 {
46     this->peso = value;
47 }
48
49 // M todos
50 void Mascota::actualizarPeso(float nuevoPeso)
51 {
52     // TODO: Implementar l gica de actualizaci n de peso
53     this->peso = nuevoPeso;
54 }
55
56 HistorialClinico* Mascota::consultarHistorialClinico()
57 {
58     // TODO: Implementar l gica de consulta de historial
59     return nullptr;
60 }
```

Listing 2: Código generado para la clase Mascota - Mascota.cpp

12.2 Anexo B: Estructura del Repositorio

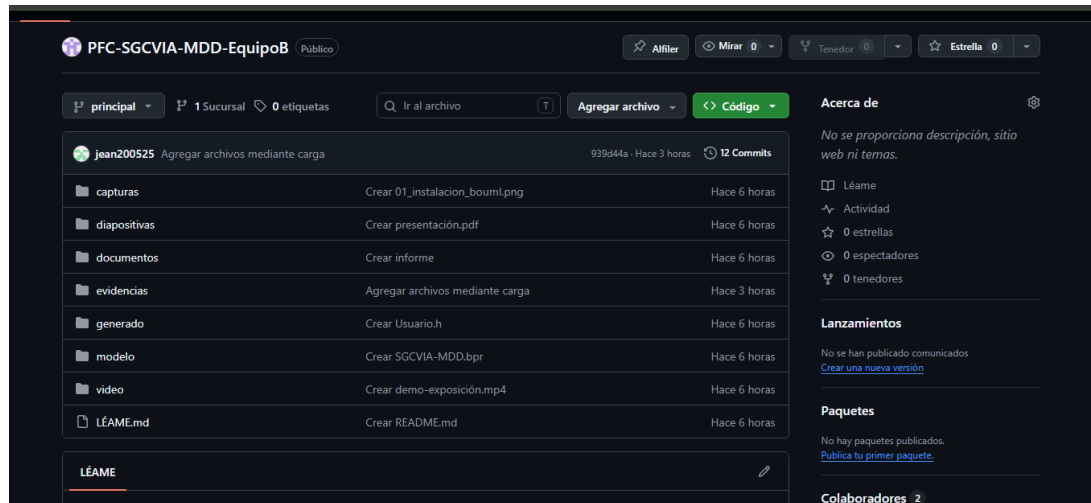


Figura 9: Estructura del Repositorio en GitHub

12.3 Anexo C: Lenguajes Soportados por BOUML

Cuadro 7: Lenguajes soportados por BOUML según captura de pantalla

Lenguaje	Soporte	Características
C++	Completo	Gestión de declaraciones/definiciones
Java	Completo	Gestión y definición por defecto
PHP	Completo	Gestión y definición por defecto
Python	Completo	Gestión y definición por defecto
IDL	Parcial	Gestión de declaraciones por defecto
MySQL	Parcial	Gestión y definición por defecto

12.4 Anexo D: Reunión del trabajo realizado

Durante la reunión se revisaron los avances obtenidos en el desarrollo del proyecto, verificando las actividades realizadas por cada integrante y el cumplimiento de las tareas asignadas. Además, se comprobó la participación de los miembros mediante los commits registrados en el repositorio de GitHub, donde cada contribución cuenta con una identidad Git verificable.

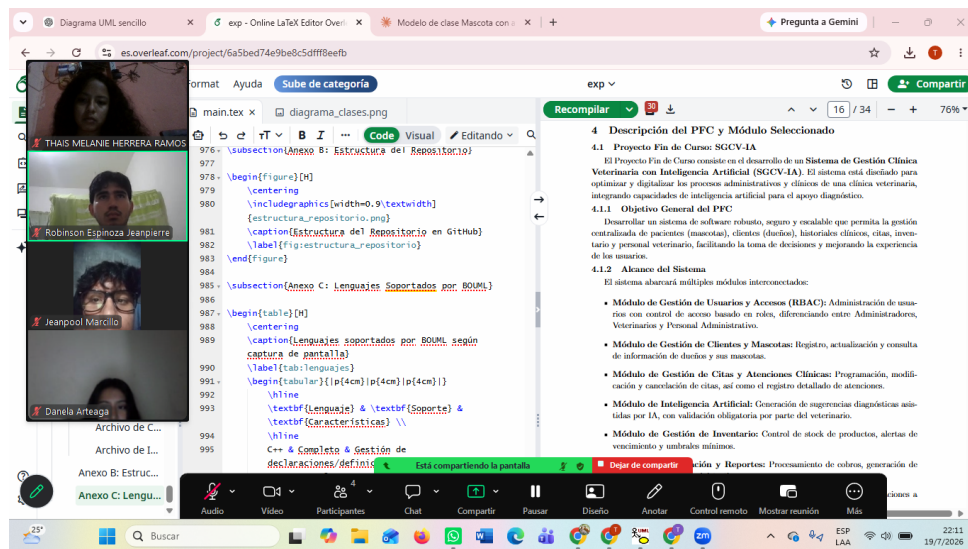


Figura 10: Captura de GitHub Insights → Contributors